第2章 詳解

02-E01 插入的完整追蹤與獨立計數

推導

j2，key1b：4>1真，[□,4a,3c,1d]+1b；i0不讀key，填得[1b,4a,3c,1d]。j3，key3c：4>3真，[1b,□,4a,1d]+3c；1>3假，填[1b,3c,4a,1d]。j4，key1d：4>1真，[1b,3c,□,4a]+1d；3>1真，[1b,□,3c,4a]+1d；1>1假，填[1b,1d,3c,4a]。

結論與檢核

h=(1,1,2)，I=4；key比較1+2+3=6；guard2+2+3=7；array writes=(1+1)+(1+1)+(2+1)=7。1b仍先於1d。最左停止b=1，公式C=4+3−1=6相符。

02-E02 內層保存條件與反例

推導

A=[2a,1b]、key1b保存後執行A[2]=A[1]，實體陣列變[2a,2a]，並非原permutation。正確表示忽略洞h=i+1：A[1..i]=B[1..i]，A[i+2..j]=B[i+1..j−1]，右段皆大於key；非洞records加key恰等於B。初始化右段空。shift將左段最後一筆複製到洞，再把新洞左移，未遺失任何邏輯record。退出時左段≤key、右段>key，填洞得到排序permutation。

結論與檢核

≥版本仍終止、排序且保存records，但等key也移右；[2a,2b]會成[2b,2a]，不穩定。若採≥，右段不變量應改成≥key，不能沿用嚴格大於敘述。

02-E03 右移恰數inversions

推導

固定右端j，已排序prefix中每筆大於S[j]者恰右移一次，不大於者不移；這些正是以j為右端的inversions。按唯一右端分割所有對，得Σh_j=I。數例h2=1，h3=0，h4=2，I=3；對為(a,b)、(a,d)、(c,d)。均勻相異排列時每對倒置機率1/2，E[I]=binom(n,2)/2=n(n−1)/4。線性期望對相依指標也成立。

結論與檢核

全等keys時嚴格>永不成立，任何identity排列都I=0，E[I]=0。不可把「均勻排列」單獨當成n(n−1)/4的充分條件；相異key前提不可省。

第2章 詳解 （續）

02-E04 以分布定義平均搜尋成本

推導

成功位置k需k次比較，條件期望Σk/n=(n+1)/2；失敗需n。全期望E[C]=s(n+1)/2+(1−s)n。代n8,s3/4：27/8+2=43/8=5.375。最差仍n；即使s=1，最後位置也可能被搜尋。

結論與檢核

對固定s∈[0,1]均為Θ(n)，但精確係數依分布而變。若成功位置不等機率，第一項改sΣkp_k，不能套均勻公式。

02-E05 穩定MERGE的完整操作

推導

(i,j)依序：(1,1)取0a→[0a]；(2,1)取1d→[0a,1d]；(2,2)等key取3b；(3,2)取3e；(3,3)取6f。此時R耗盡，接7c。輸出[0a,1d,3b,3e,6f,7c]。比較5次，copy進L/R共6，寫回6。

結論與檢核

等key改先取R會得[0a,1d,3e,3b,6f,7c]，仍有序且是permutation，故符合未要求穩定的排序；不符合穩定排序。五次比較達m−1上界。

02-E06 Sentinel與別名不是免費假設

推導

L加sentinel後[M_real,M_s]，R為[M_real,M_real,M_s]。固定三次輸出且等值取左：第一取L[1]，第二取L[2]哨兵，第三欲讀L[3]越界。標記值與合法key相同破壞了「哨兵最後才會選到」。view反例：A=[2,4,1,3]，首選右半1寫A[1]，使L的首值從2變1；下次便錯取已覆寫的1，結果資料重複。

結論與檢核

獨立copy兩半並使用長度檢查；任一耗盡後只讀另一半，不需域外值。若仍用哨兵，須tagged值嚴格大於所有real records，不能只是最大合法key。

02-E07 奇數長度的遞迴與峰值空間

推導

根(1,3,5)，左(1,2,3)，其左(1,1,2)合[2b,6a]；再合左三筆[2b,5c,6a]。右(4,4,5)合[1d,4e]。根合[1d,2b,4e,5c,6a]。四次MERGE大小2,3,2,5，copy進buffers=12筆，回寫=12筆。

第2章 詳解 （續）

結論與檢核

最大單次buffer合5筆。子buffers合完即釋放，故峰值buffer=5；累計12不是峰值。遞迴最深同時4個呼叫（長5→3→2→1），一般stack Θ(lg n)，record buffer一般Θ(n)。

02-E08 兩個遞迴式的完整解

推導

h=lg n。U每個internal depth成本5n，共h層，葉3n，得U=5n lg n+3n；n8得120+24=144。W深度ℓ成本2^ℓ(n/2^ℓ−1)=n−2^ℓ，Σℓ0..h−1得n lg n−(2^h−1)=n lg n−n+1，n8得17。也可把公式代入遞迴式並核n1基例，完成歸納。

結論與檢核

MERGE的比較依耗盡位置而變，W只是所有合併取m−1的最差遞迴計數；有序八筆的各層比較4+4+4=12，不是17。buffer複製與回寫仍每internal層各8，不能由比較減少就判全部工作減少。

02-E09 非2冪次的Θ界

推導

每次ceil/floor平分使深度ℓ區間大小在floor(n/2^ℓ)與ceil(n/2^ℓ)之間，直到大小1停止。height≤ceil(lg n)；任一深度未終止區間互斥，總大小≤n，每internal成本≤c_high大小，故internal總O(n lg n)。當ℓ<floor(lg n)，最小區間仍≥2，所有區間皆internal且覆蓋n筆，該層成本≥c_low n，共floor(lg n)層。n≥2有floor(lg n)≥lg n/2，得Ω(n lg n)。

結論與檢核

恰n個單筆葉各Θ(1)，共Θ(n)，不改主階；n0/1獨立Θ(1)。證明以真區間大小與同層成本給上下界，未假設未證的時間單調性。

02-E10 模型與早停變體

推導

已排序時insertion沒有shift，Θ(n)；標準MERGE仍對每個internal區間copy與回寫，Θ(n lg n)。變體先遞迴排好兩半；若左最大A[q]≤右最小A[q+1]，兩半串接即整段有序，相等的左records原本就先於右records，故仍穩定。已排序輸入每節點均略過MERGE，只做常數檢查；二元遞迴樹n葉、n−1 internal nodes，時間Θ(n)。最差仍O(n lg n)，略過只刪工作，反例交錯可保留合併成本。

第2章 詳解 （續）

結論與檢核

若比較最差需Θ(L)，key比較計數須乘其實際成本；不可直接把每次guard視為常數。若record只存reference，搬移仍可O(1)，所以也不能不分操作就把所有步數一概乘L當精確時間。

原創驗算程式附錄: insertion_sort.py

```
"""Trace insertion sort and verify the inversion count."""

def insertion_trace(values):
    a = list(values)
    moves = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
            moves += 1
        a[j + 1] = key
    return a, moves


def inversions(values):
    return sum(values[i] > values[j] for i in range(len(values)) for j in range(i + 1,
len(values)))


sample = [7, 2, 6, 1]
sorted_values, moves = insertion_trace(sample)
assert sorted_values == sorted(sample)
assert moves == inversions(sample) == 5
print(sorted_values, moves)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第2章 詳解 （續）

原創驗算程式附錄: sorting_models.py

```
"""Finite independent sorting/count checks; universal arguments are in the chapter."""
from itertools import product,permutations
def insertion(a):
 a=list(a); shifts=comp=guard=writes=0
 for j in range(1,len(a)):
  key=a[j];i=j-1
  while True:
   guard+=1
   if i<0:break
   comp+=1
   if not a[i][0]>key[0]:break
   a[i+1]=a[i];writes+=1;shifts+=1;i-=1
  a[i+1]=key;writes+=1
 return a,(shifts,comp,guard,writes)
def merge_sort(a):
 a=list(a);copies=writes=comps=peak=0
 def go(p,r):
  nonlocal copies,writes,comps,peak
  if r-p<=1:return
  q=p+(r-p+1)//2;go(p,q);go(q,r)
  left=a[p:q];right=a[q:r];copies+=r-p;peak=max(peak,r-p)
  i=j=0;out=[]
  while i<len(left) and j<len(right):
   comps+=1
   if left[i][0]<=right[j][0]:out.append(left[i]);i+=1
   else:out.append(right[j]);j+=1
  out+=left[i:]+right[j:];a[p:r]=out;writes+=r-p
 go(0,len(a));return a,(copies,writes,comps,peak)
if __name__=='__main__':
 count=0
 for n in range(7):
  for keys in product(range(3),repeat=n):
   a=list(zip(keys,range(n)));expected=sorted(a,key=lambda x:x[0])
   inv=sum(keys[i]>keys[j] for i in range(n) for j in range(i+1,n))
   b,c=insertion(a);assert b==expected and c[0]==inv
   assert c[2]==inv+max(0,n-1) and c[3]==c[2]
   b,d=merge_sort(a);assert b==expected and d[0]==d[1]
   count+=1
 assert insertion([(4,'a'),(1,'b'),(3,'c'),(1,'d')])[1]==(4,6,7,7)
 assert merge_sort(list(zip([6,2,5,1,4],'abcde')))[1][0:2]==(12,12)
 assert merge_sort(list(zip([1,2,3,4,5,6],range(6))))[1][0:2]==(16,16)
 assert merge_sort(list(zip(range(8),range(8))))[1][2]==12
 assert max(merge_sort(list(zip(p,range(6))))[1][2] for p in permutations(range(6)))==11
 print(f'PASS: {count} exhaustive repeated-key arrays n<=6; trace counts, stable outputs,
inversion identity; finite only')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第2章 詳解 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study